Shri Vile Parle Kelavani Mandal's
DWARKADAS J.SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous College Affiliated to the University of Mumbai)

AI-Based Music Ornament Detection for Indian Classical Music

Submitted in partial fulfillment of the requirement of the degree of

Bachelor of Technology in
Computer Science and Engineering (Data Science)
Department of Computer Science and Engineering (Data Science)

By

Dev Shah	**60009240037**
Yug Doshi	**60009240066**
Yashraj Bilakhiya	**60009240127**
Aarav Dave	**60009240096**
Karan Bhanushali	**60009240149**

Under the guidance of

Dr. Nilesh Marathe

Name of Guide

A.Y. 2026-27

CERTIFICATE

This is to certify that the project entitled, **AI-Based Music Ornament Detection for Indian Classical Music** is a bonafide work of **Dev Shah (SAP ID: 60009240037)**, **Yug Doshi (SAP ID: 60009240066)**, **Yashraj Bilakhiya (SAP ID: 60009240127)**, **Aarav Dave (SAP ID: 60009240096)**, **Karan Bhanushali (SAP ID: 60009240149)** submitted in the partial fulfillment of the requirement for the award of the Bachelor of Technology in Computer Science and Engineering (Data Science).

Dr. Nilesh Marathe
Name of the Guide

Dr. Kriti Srivastava
Head of the Department

Dr. Hari Vasudevan
Principal

Place:

Date:

DECLARATION

We declare that this written submission represents our ideas in our own words and where others ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that, we have adhered to all the principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources, which have thus not been properly cited or from whom proper permission has not been taken, when needed.

_____

Dev Shah (SAP ID: 60009240037)

_____

Yug Doshi (SAP ID: 60009240066)

_____

Yashraj Bilakhiya (SAP ID: 60009240127)

_____

Aarav Dave (SAP ID: 60009240096)

_____

Karan Bhanushali (SAP ID: 60009240149)

Place:

Date:

APPROVAL SHEET

The Project entitled, **AI-Based Music Ornament Detection for Indian Classical Music**, submitted by **Dev Shah (SAP ID: 60009240037)**, **Yug Doshi (SAP ID: 60009240066)**, **Yashraj Bilakhiya (SAP ID: 60009240127)**, **Aarav Dave (SAP ID: 60009240096)**, **Karan Bhanushali (SAP ID: 60009240149)** is approved for the award of the Bachelor of Technology in Computer Science and Engineering (Data Science).

Examiners:

1. **----------**
2. **----------**

Place:

Date:

Acknowledgement

We are deeply grateful to everyone who contributed to the progress of our project. First and foremost, we extend our heartfelt thanks to our project guide, Dr. Nilesh Marathe, Department of Computer Science and Engineering (Data Science), whose mentorship, expertise and insightful feedback shaped the direction of this work.

We also express our appreciation to the faculty members and to our Head of Department, Dr. Kriti Srivastava, Department of Computer Science and Engineering (Data Science), Dwarkadas J. Sanghvi College of Engineering, for their continuous encouragement and for fostering an environment of learning and innovation.

We thank our Principal, Dr. Hari Vasudevan, for making the college library and research resources accessible to us. We also acknowledge the creators of the Raga Ornamentation Detection (ROD) dataset, whose publicly released recordings and annotations made this work possible.

Name of the Students:

_____
Dev Shah

_____
Yug Doshi

_____
Yashraj Bilakhiya

_____
Aarav Dave

_____
Karan Bhanushali

Table of Contents

List of Figures **2**

List of Tables **4**

Abstract **5**

1. Introduction **6**

1.1 Background 6

1.2 Motivation 8

2. Literature Review **9**

2.1 Literature Survey 9

2.2 Study of Existing Systems 10

2.3 Analysis of Literature Survey 11

2.4 Parameters for Evaluation 13

2.5 Research Gaps 14

3. Problem Definition and Objectives **16**

3.1 Problem Definition 16

3.2 Problem Objectives 16

4. Proposed Solution **18**

4.1 Overview of the Solution 18

4.2 Key Modules 19

4.3 System Architecture 26

4.4 Implementation 27

5. Plan for Next Semester **45**

References **46**

List of Figures

Fig. 1.1 Schematic pitch trajectories of the six ornaments studied (illustrative shapes, not measured data) 7

Fig. 4.1 Overview of the proposed ornament recognition pipeline 18

Fig. 4.2 Temporal segmentation into frames of 100 samples with a stride of 50 (50 % overlap) 21

Fig. 4.3 Architecture of the ornament-specific 1D CNN classifier 22

Fig. 4.4 Training workflow of an ornament-specific CNN classifier 24

Fig. 4.5 Testing and inference workflow on an unseen recording 25

Fig. 4.6 System architecture of the proposed ornament recognition framework 26

Fig. 4.7 Distribution of the ROD dataset: (a) recorded duration per rāga, (b) recordings per singer 28

Fig. 4.8 Annotated duration ranges of the six ornaments compared with the one-second CNN frame 29

Fig. 4.9 Sample recording (001_bageshree_1_Alankar_1_Swar.wav): (a) waveform, (b) CREPE F0 contour, (c) voicing confidence 31

Fig. 4.10 Frequency, velocity and acceleration computed with NumPy for a 7-second excerpt of the sample recording 32

Fig. 4.11 Class balance of the Meend training frames derived from pos_weight = 7.19 33

Fig. 4.12 Experimental framework and decision rules for hypotheses H1 and H2 33

Fig. 4.13 Comparison of input representations for hypothesis H1 36

Fig. 4.14 Incremental change in each metric contributed by velocity and by acceleration 38

Fig. 4.15 Macro F1 of non-learning baselines compared with the CNN models 39

Fig. 4.16 Frame-wise Meend probability on the test recording under two standardisation settings 40

Fig. 4.17 Number of frames detected as Meend under different preprocessing settings 41

Fig. 4.18 Deployment architecture of the prototype 42

Fig. 4.19 Dashboard of the prototype analysing a ROD recording 43

Fig. 4.20 Detection panel of the prototype showing frame-wise probabilities on the test recording 44
Fig. 5.1 Timeline of the work planned for the next semester 45

List of Tables

Table 1.1 Indian classical vocal ornaments considered in this project 7

Table 2.1 Analysis of literature survey 11

Table 2.2 Parameters for evaluation 13

Table 2.3 Research gaps identified from the literature 14

Table 4.1 Configuration of the 1D CNN classifier 22

Table 4.2 Summary of the ROD dataset [3], [26] 27

Table 4.3 Duration of ROD recordings per rāga [3] 28

Table 4.4 Duration rules used for annotating each ornament [3] 29

Table 4.5 Measured properties of the sample recording 30

Table 4.6 Formulation of hypothesis H1 34

Table 4.7 Formulation of hypothesis H2 34

Table 4.8 Training and testing configuration 35

Table 4.9 Effect of the input representation on classification performance (mean ± s.d.) 36

Table 4.10 Incremental change in each metric when pitch dynamics are added 37

Table 4.11 Macro F1 of non-learning baselines and CNN models 38

Table 4.12 Highest-probability frames of the test recording 40

Table 4.13 Summary of the result analysis 41

Abstract

Ornaments such as Meend, Gamak, Murki, Kan, Andolan and Nyas Swar define the expressive character of Indian classical vocal music, yet their continuous, microtonal pitch movements are poorly captured by note-based or spectrogram-based representations, which makes their automatic recognition an open problem in Music Information Retrieval. The AIBased Ornament Detector addresses this problem by recognising the six ornaments directly from the pitch contour of a vocal recording. The audio is loaded and converted to mono with Librosa, and its fundamental frequency is estimated every 10 ms with the pretrained CREPE network. NumPy then derives the velocity and acceleration of the pitch contour, so that each time step is described by its pitch level, the speed of its movement and the change in that speed. The three-channel sequence is standardised with training-set statistics, divided into onesecond frames of 100 time steps with 50 % overlap, and classified by six ornament-specific one-dimensional Convolutional Neural Networks, each producing a sigmoid probability that is compared with an ornament-specific threshold. The system is developed on the expertannotated Raga Ornamentation Detection (ROD) dataset of 212 Hindustani vocal recordings (4.08 hours) by two singers across four rāgas. Two hypotheses are tested: that pitch dynamics add information beyond pitch alone, and that a temporal CNN learns ornament patterns better than a non-learning baseline. Adding velocity and acceleration raised the macro F1-score from 0.491 to 0.821, and the resulting model exceeded the best non-learning baseline by 0.321, while accuracy alone proved misleading on the imbalanced ornament data. A working Meend detection prototype, consisting of a FastAPI backend and a React dashboard, demonstrates the complete pipeline from an uploaded recording to timestamped frame-level predictions, and the next stage extends the detector to all six ornaments with formal evaluation on unseen singers and rāgas.

Keywords: ornament detection, Meend, CREPE, pitch contour, 1D CNN.

1. Introduction

Indian classical music is a highly expressive musical tradition in which pitch movements, microtonal variations and melodic ornamentation play a central role. The identity of a musical phrase is determined not only by the notes being performed but also by the manner in which a singer moves between and around those notes. Studies of ornamentation in Hindustani vocal music highlight the importance of these expressive pitch movements in performance [1], [2].

These expressive pitch movements are known as ornaments or *alankars*. Important ornaments in Indian classical vocal music include Meend, Gamak, Murki, Kan (Kanswar), Andolan and Nyas Swar. Each ornament has a characteristic pattern of pitch movement and duration: Meend is a smooth glide between notes, whereas Gamak is a rapid, forceful oscillation. These differences make ornament recognition an interesting problem in Music Information Retrieval (MIR) [1]–[3].

This project develops an automated system for recognising these six ornaments using a Convolutional Neural Network (CNN). Rather than depending only on isolated pitch values, the proposed system analyses continuous pitch behaviour through the fundamental frequency and its temporal variations. Frequency, velocity and acceleration are used to represent the movement of pitch, allowing the CNN to learn the patterns associated with each ornament category.

1.1 Background

Music Information Retrieval involves extracting meaningful information from audio recordings using computational and signal-processing techniques. Common MIR tasks include music classification, melody analysis, pitch estimation, transcription and singing-technique recognition. Research on singing-technique detection shows how computational models can identify vocal characteristics directly from recordings [9]–[11].

In Indian classical music, ornamentation is especially important because melodic expression depends on continuous pitch transitions and subtle variations. These movements differ in speed, direction, smoothness and oscillatory behaviour. Consequently, analysing only the occurrence of individual notes is not sufficient to distinguish between ornaments [1], [2], [4].

The fundamental frequency (F0) is the primary frequency of a voiced sound and is closely related to its perceived pitch. Tracking F0 over time produces a pitch contour that describes how the singer's pitch changes throughout a recording. CNN-based pitch estimators such as CREPE estimate F0 directly from the waveform with high accuracy [12]. The shape and temporal behaviour of this contour carry the information needed to analyse ornaments.

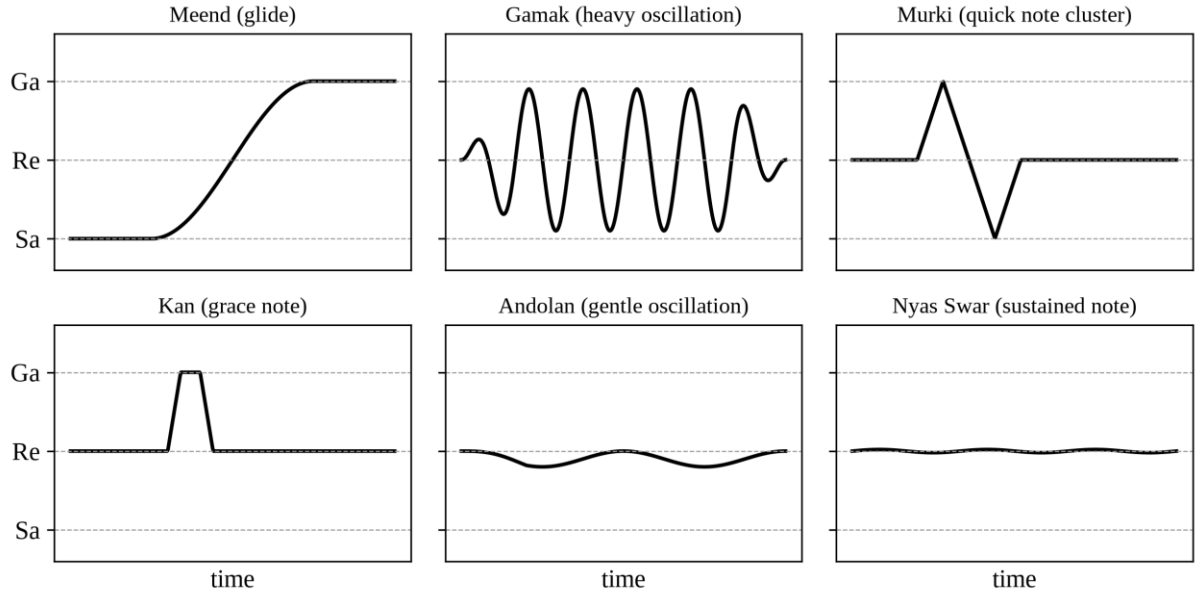


Fig. 1.1 Schematic pitch trajectories of the six ornaments studied (illustrative shapes, not measured data)

Fig. 1.1 shows the characteristic pitch movement of each ornament relative to the swaras Sa, Re and Ga.

Table 1.1 Indian classical vocal ornaments considered in this project

Ornament	Pitch behaviour	Typical contour signature
Meend	Smooth, sustained glide from one note to another	Monotonic rise or fall with low acceleration
Gamak	Fast, heavy oscillation between two notes	Large periodic swings; high velocity and acceleration
Murki	Quick, light cluster of two or three neighbouring notes	Short rapid steps within a fraction of a second
Kan (Kanswar)	Brief touch of an adjacent grace note	Very short spike before or after a main note
Andolan	Gentle, slow oscillation around a note	Low-amplitude, low-frequency sway

Ornament	Pitch behaviour	Typical contour signature
Nyas Swar	Emphasised, sustained resting note	Nearly flat contour; velocity close to zero

Table 1.1 describes the pitch behaviour and the typical contour signature that distinguishes each of the six ornaments.

1.2 Motivation

Recognising Indian classical ornaments manually requires musical training, listening experience and an understanding of subtle pitch movements. Automating this process can support music education, computational musicology, digital archiving and performance analysis. For example, automatic feedback on ornament execution can assist students practising alankars, and ornament annotations can enrich large archives of recorded concerts.

Different ornaments may share overlapping pitch ranges yet differ in *how* the pitch changes over time. The proposed approach therefore considers not only the fundamental frequency but also its temporal derivatives. Frequency, velocity and acceleration provide complementary information about the position, speed and curvature of the pitch trajectory. A single, consistent CNN framework is used to learn these patterns for all six ornaments, so that the same methodology can be applied systematically across ornament categories.

2. Literature Review

2.1 Literature Survey

Research in Indian classical music analysis has explored several approaches to understanding pitch movement, ornamentation and singing techniques. These include traditional pitch-contour analysis, handcrafted feature extraction, spectrogram-based deep learning and temporal modelling.

2.1.1 Pitch-Contour-Based Ornament Analysis

Pratyush [1] studied the analysis and classification of ornaments in North Indian classical music, showing that pitch-based representations are well suited to describing ornamentation because pitch contours preserve the continuous transitions that define ornaments. Banerjee et al. [2] examined ornamentation in Hindustani vocal music from a musicological and signal-analysis perspective. Gupta and Rao [4] used pitch-contour comparison to objectively assess ornament execution, and Miryala et al. [6] detected andolan and meend by fitting line segments to the pitch curve and matching them to templates.

2.1.2 Deep Learning for Ornament and Singing-Technique Recognition

Kumar, Singh and Arora [3] introduced the Raga Ornamentation Detection (ROD) dataset and an encoder–decoder Temporal Convolutional Network that detects six ornaments from chromagram inputs, outperforming a CRNN baseline. Yamamoto et al. explored singing-technique detection with deep learning [9], deformable CNNs with imbalance-aware feature learning [10] and characteristics-aware network customisation [11]. Deep learning has also been applied to rāga recognition in Indian classical music [7], [23], [24].

2.1.3 CNN-Based Audio Classification

Piczak [15] demonstrated CNNs for environmental sound classification, and Salamon and Bello [16] showed that deep CNNs combined with data augmentation improve robustness. Convolutional recurrent architectures have been applied to sound-event detection [17], [18], and sound-event detection has been formalised as a general framework for locating events in time [22].

2.1.4 Temporal Modelling

Lea et al. [19], [20] introduced Temporal Convolutional Networks for action segmentation and detection, and Bai et al. [21] showed that generic convolutional sequence models are competitive with recurrent networks. These studies motivate the use of one-dimensional convolutions over time to model how a pitch contour evolves.

2.2 Study of Existing Systems

Existing approaches for music and singing-technique analysis can broadly be grouped into four categories.

1. **Traditional pitch-based approaches** analyse pitch contours using signal-processing techniques and manually designed rules [1], [4], [6]. They provide meaningful information about pitch transitions, but rule-based methods require additional feature engineering for every new ornament category and do not scale easily across singers and styles.
2. **Spectrogram-based CNN approaches** convert audio into time–frequency representations and learn patterns with CNNs [15]–[18]. They are effective for audio analysis, but a spectrogram or chromagram does not directly represent the fundamental-frequency trajectory as a continuous sequence of pitch values; pitch is discretised into frequency bins [3].
3. **Singing-technique classification approaches** apply deep learning, including deformable CNNs and characteristics-aware networks, to identify vocal techniques [9]–[11]. They demonstrate the potential of learned representations but are designed for Western and popular singing techniques rather than Indian ornaments.
4. **Temporal convolutional approaches** learn patterns across sequential data and have been used for action segmentation and ornament event detection [3], [19]–[21]. Their effectiveness depends on the input representation and on how well it reflects the pitch characteristics being analysed.

Proposed approach. This project explores a CNN that operates directly on the continuous fundamental-frequency contour and its temporal variations. Frequency, velocity and acceleration describe pitch behaviour, allowing the model to learn patterns associated with Meend, Gamak, Murki, Kan (Kanswar), Andolan and Nyas Swar.

2.3 Analysis of Literature Survey

The reviewed literature is summarised below.

Table 2.1 Analysis of literature survey

No.	Authors (Year)	Title	Source	Methodology
1	Pratyush (2010)	Analysis and Classification of Ornaments in North Indian (Hindustani) Classical Music	Master's thesis, Universitat Pompeu Fabra [1]	F0-based pitch-contour analysis with rule-based classification of vocal ornaments.
2	K. Banerjee et al. (2024)	Ornamentation in Hindustani Vocal Music	Proc. FRSM 2023, Springer [2]	Analyses ornaments in Hindustani vocal performances using audio segments and pitch characteristics.
3	S. Kumar, P. Singh, V. Arora (2026)	Recognizing Ornaments in Vocal Indian Art Music With Active Annotation	IEEE/ACM Trans. Audio, Speech and Language Processing [3]	Introduces the ROD dataset and an encoder–decoder TCN on chromagrams for ornament event detection.
4	C. Gupta, P. Rao (2012)	Objective Assessment of Ornamentation in Indian Classical Singing	CMMR 2011, LNCS 7172, Springer [4]	Compares pitch-contour segments of ornaments between a reference and a student performance.
5	A. Vidwans, K. K. Ganguli, P. Rao (2012)	Classification of Indian Classical Vocal Styles from Melodic Contours	Proc. 2nd CompMusic Workshop [5]	Extracts melodic-contour features to separate Hindustani and Carnatic vocal styles.
6	S. S. Miryala et al. (2013)	Automatically Identifying Vocal Expressions for Music Transcription	Proc. ISMIR [6]	Line-fitting of the pitch curve and template matching to detect andolan and meend without training data.
7	S. Gulati et al. (2016)	Time-Delayed Melody Surfaces for Rāga Recognition	Proc. ISMIR [7]	Builds tonic-normalised melody surfaces from pitch contours for rāga classification.
8	S. Giraldo, R. Ramírez (2018)	A Machine Learning Approach to Ornamentation Modeling and Synthesis in Jazz Guitar	Machine Learning and Music Generation, Routledge [8]	Machine-learning models for describing and synthesising ornamentation in jazz guitar.
9	Y. Yamamoto, J. Nam, H. Terasawa (2022)	Analysis and Detection of Singing Techniques in Repertoires of J-Pop Solo Singers	Proc. ISMIR [9]	Audio features and neural classifiers for frame-wise singing-technique detection.

No.	Authors (Year)	Title	Source	Methodology
10	Y. Yamamoto et al. (2022)	Deformable CNN and Imbalance-Aware Feature Learning for Singing Technique Classification	arXiv:2206.12230 [10]	Deformable convolutions and imbalance-aware learning on spectrogram inputs.
11	Y. Yamamoto et al. (2023)	PrimaDNN: A Characteristics-Aware DNN Customization for Singing Technique Detection	Proc. EUSIPCO [11]	Customises a DNN using knowledge of singing-technique characteristics.
12	J. Kim, J. Salamon, P. Li, J. P. Bello (2018)	CREPE: A Convolutional Representation for Pitch Estimation	Proc. IEEE ICASSP [12]	Deep CNN that estimates F0 directly from 1024-sample waveform frames.
13	J. Wilkins et al. (2018)	VocalSet: A Singing Voice Dataset	Proc. ISMIR [13]	Singing-voice dataset of multiple singers and vocal techniques.
14	K. J. Piczak (2015)	Environmental Sound Classification with Convolutional Neural Networks	IEEE MLSP [15]	CNN on log-mel spectrograms for environmental sound classification.
15	J. Salamon, J. P. Bello (2017)	Deep Convolutional Neural Networks and Data Augmentation for Environmental Sound Classification	IEEE Signal Processing Letters [16]	Deep CNN with audio data augmentation for robust sound recognition.
16	E. Çakır et al. (2017)	Convolutional Recurrent Neural Networks for Polyphonic Sound Event Detection	IEEE/ACM TASLP [17]	Combines convolutional and recurrent layers for sound-event detection.
17	C.-C. Kao et al. (2018)	R-CRNN: Region-Based Convolutional Recurrent Neural Network for Audio Event Detection	arXiv preprint [18]	Region-based CRNN that localises audio events in time.
18	C. Lea et al. (2016)	Temporal Convolutional Networks: A Unified Approach to Action Segmentation	ECCV Workshops [19]	Temporal convolutions to capture long-range dependencies in sequences.
19	S. Bai, J. Z. Kolter, V. Koltun (2018)	An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling	arXiv:1803.01271 [21]	Shows simple temporal CNNs match or outperform recurrent networks on many sequence tasks.

Table 2.1 lists the authors, title, source and methodology of each of the nineteen reviewed works.

The reviewed literature shows that pitch analysis and deep learning are both relevant to recognising musical and vocal characteristics. Traditional pitch-contour approaches provide meaningful information about continuous pitch transitions and ornament characteristics [1], [2], [4], but rely on manually defined rules. Deep learning approaches learn patterns from data, and CNN-based methods are effective in audio classification and sound analysis [15]–[18]; singing-technique research further demonstrates their potential for vocal characteristics [9]–[11], while temporal convolutional models learn patterns across sequences [19]–[21].

An important consideration is the representation supplied to the model. Indian classical ornaments are characterised by changes in pitch over time, including variations in slope, curvature and oscillatory behaviour. A representation that explicitly includes pitch dynamics can therefore provide useful information for distinguishing ornaments. Based on these observations, this project uses the fundamental frequency together with its velocity and acceleration, and a CNN learns the patterns associated with each ornament category.

2.4 Parameters for Evaluation

Each ornament-specific classifier is evaluated with the parameters listed below. Accuracy, precision, recall and F1-score are computed from the confusion matrix of true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN). These metrics are commonly used in classification and audio-classification studies [15], [16], [22].

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1 = 2 \cdot Precision \cdot Recall / (Precision + Recall)$$

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) \quad (2.4)$$

For a binary classifier, the macro F1-score averages the F1-scores of the positive and negative classes, so the minority ornament class carries equal weight:

$$Macro\ F1 = (F1(present) + F1(absent)) / 2$$

Table 2.2 Parameters for evaluation

No.	Evaluation parameter	Purpose

No.	Evaluation parameter	Purpose
1	Accuracy	Proportion of correctly classified positive and negative samples for each ornament classifier.
2	Precision	How many samples predicted as positive for an ornament actually belong to that ornament.
3	Recall	How many actual samples of an ornament are correctly identified by its classifier.
4	F1-score (and macro F1)	Balanced combination of precision and recall; macro F1 averages over classes and is robust to class imbalance.
5	Confusion matrix	TP, TN, FP and FN counts for each binary classifier; reveals which ornaments are confused.
6	Training and validation loss	Monitors learning behaviour and helps identify overfitting.
7	Prediction probability and threshold	Sigmoid output probability compared with the ornament-specific decision threshold τ.

Table 2.2 lists the seven evaluation parameters and the purpose of each in assessing an ornament classifier.

2.5 Research Gaps

The literature review highlights the following research gaps.

Table 2.3 Research gaps identified from the literature

No.	Research gap	Description	Supporting work
1	Dependence on handcrafted analysis	Traditional pitch-based approaches rely on predefined rules and manually selected features, creating scope for models that learn relevant patterns automatically.	[1], [2], [4], [6]
2	Continuous pitch representation	Spectrograms and chromagrams discretise pitch into bins, whereas a direct F0 representation preserves the continuous trajectory of the voice.	[2], [3], [15]–[18]
3	Ornament-specific classification	Existing deep models detect ornaments jointly from spectral inputs; independent binary CNN classifiers on pitch features remain unexplored.	[3], [9]–[11]
4	Temporal pitch dynamics	F0 values alone may not capture differences between ornaments; velocity and acceleration describe the rate and variation of pitch movement.	[1], [2], [4]

Table 2.3 summarises the four research gaps and the works that support each of them.

Our Contribution

This project contributes a CNN-based framework for recognising six Indian classical vocal ornaments: Meend, Gamak, Murki, Kan (Kanswar), Andolan and Nyas Swar. The approach uses the continuous fundamental-frequency contour and derives velocity and acceleration to represent the temporal behaviour of pitch. These features are standardised, segmented into overlapping one-second frames and classified by one compact 1D CNN per ornament. The contribution of pitch dynamics and of temporal learning is examined through two formally stated hypotheses (Section 4.4), and an end-to-end prototype for Meend detection has been implemented.

3. Problem Definition and Objectives

3.1 Problem Definition

Indian classical vocal music involves expressive melodic ornaments characterised by continuous pitch movements, such as transitions, oscillations, rapid note clusters and sustained notes. Recognising these ornaments from audio is challenging because different ornaments may share overlapping pitch ranges while differing in their temporal patterns, so note-based representations alone cannot capture them adequately.

The problem addressed in this project is to develop an automated framework that recognises six Indian classical vocal ornaments, namely Meend, Gamak, Murki, Kan (Kanswar), Andolan and Nyas Swar, from audio recordings.

The system accepts a WAV recording and uses Librosa for audio loading and preprocessing. CREPE then estimates the fundamental frequency (F0) every 10 ms. NumPy derives three features from the pitch sequence: frequency, velocity and acceleration, representing the pitch value, its rate of change and the rate of change of its velocity. The three-channel representation is standardised and divided into overlapping frames of 100 time steps with an overlap of 50 time steps. Each frame is supplied to a One-Dimensional Convolutional Neural Network (1D CNN) that learns the temporal patterns associated with vocal ornaments.

The system uses six ornament-specific binary classifiers, one CNN per ornament. Each model outputs a probability through a sigmoid activation, and an ornament-specific decision threshold τ determines whether the ornament is present in the frame. The central problem is therefore to build a pitch-feature-based CNN framework that captures the characteristic temporal behaviour of each ornament and performs reliable ornament-specific binary classification.

3.2 Problem Objectives

The main objectives of the project are:

1. To study the musical characteristics and pitch behaviour of six Indian classical vocal ornaments: Meend, Gamak, Murki, Kan (Kanswar), Andolan and Nyas Swar.

2. To extract a reliable pitch representation (F0, velocity and acceleration) from vocal recordings using CREPE and NumPy.
3. To design and develop a 1D CNN that learns temporal patterns from the three-channel pitch-feature representation.
4. To formulate and test hypotheses on the value of pitch dynamics (H1) and of temporal learning (H2).
5. To evaluate the classifiers using accuracy, precision, recall, F1-score, confusion matrices and training and validation behaviour.
6. To develop a framework and application that support automated analysis and recognition of ornaments in recordings.

4. Proposed Solution

4.1 Overview of the Solution

The proposed system recognises Indian classical vocal ornaments by analysing the continuous movement of the fundamental frequency (F0) over time. Previous research has shown, through pitch-contour analysis and musicological observation, that pitch behaviour is the key to understanding ornaments [1], [2], [4]. Instead of relying solely on spectrogram features, the system therefore learns temporal patterns directly from pitch-related features, building on the success of deep learning and CNNs for singing-technique recognition and audio classification [9]–[11], [15], [16].

A WAV recording is loaded and prepared with Librosa and passed to CREPE, a CNN that estimates pitch directly from the time-domain waveform [12]. F0 is extracted every 10 ms to form a continuous pitch contour, from which NumPy derives velocity and acceleration. The three features are standardised with the mean and standard deviation of the training data, divided into overlapping 100-step frames with a 50-step overlap, and classified by six ornament-specific 1D CNNs.

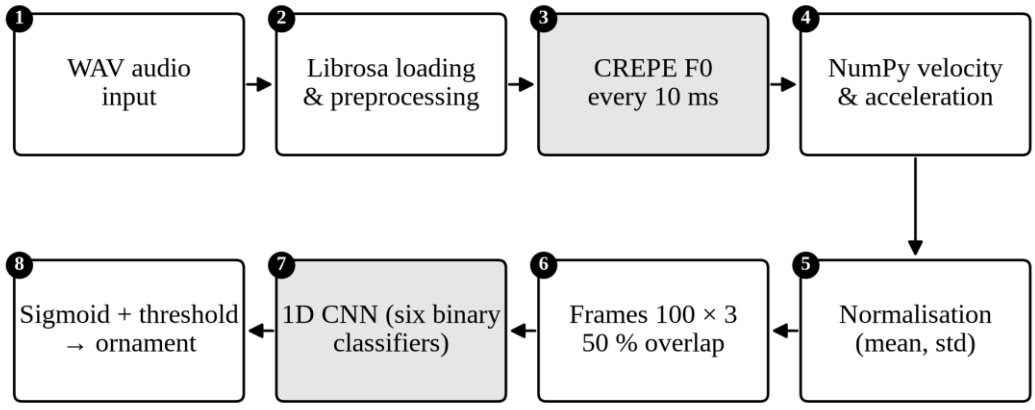


Fig. 4.1 Overview of the proposed ornament recognition pipeline

Fig. 4.1 shows the eight processing stages, from the input WAV file to the sigmoid decision of the ornament classifiers.

1. **WAV audio input:** the vocal recording to be analysed.
2. **Librosa loading & preprocessing:** loads the waveform and converts it to mono.

3. **CREPE F0 every 10 ms:** estimates the fundamental frequency and its confidence.
4. **NumPy velocity & acceleration:** computes the first and second time derivatives of F0.
5. **Normalisation (mean, std):** standardises each feature channel with training statistics.
6. **Frames 100 × 3, 50 % overlap:** cuts the feature sequence into one-second overlapping frames.
7. **1D CNN (six binary classifiers):** scores every frame with one CNN per ornament.
8. **Sigmoid + threshold → ornament:** turns each score into a probability and a present/absent decision.

4.2 Key Modules

The proposed system consists of the following modules.

4.2.1 Audio Input and Preprocessing

The system accepts a recording in WAV format. Librosa [14] loads the file and converts it into a waveform together with its sampling rate; stereo recordings are converted to mono. Librosa is responsible for audio loading and preprocessing, whereas CREPE is used specifically for estimating the fundamental frequency.

4.2.2 Fundamental Frequency Extraction

The fundamental frequency represents the perceived pitch of the voice and is the most important characteristic for identifying ornaments. The waveform is passed to CREPE (Convolutional Representation for Pitch Estimation), a pretrained deep network that estimates F0 directly from the time-domain signal [12]. CREPE resamples the audio to 16 kHz, analyses 1024-sample windows and returns an F0 estimate and a voicing confidence every 10 ms, producing a time-dependent pitch contour:

$$F0 = [\, f1, f2, f3, \ldots, fn \,]$$

where fi is the estimated fundamental frequency at the i-th time step. Samples with low confidence (for example, silence or tanpura-only regions) are treated as unvoiced.

4.2.3 Pitch Feature Representation

Three complementary features are derived from the F0 sequence. Together they capture the pitch value and its temporal movement, which are needed to distinguish ornaments.

- **Frequency (F0):** the fundamental frequency estimated by CREPE at each time step; it describes the pitch level and is the primary feature.
- **Velocity:** the rate of change of pitch over time, indicating how quickly the pitch rises or falls.
- **Acceleration:** the rate of change of velocity, capturing changes in the speed of pitch movement that characterise oscillating and curved ornaments.

Velocity and acceleration are defined as:

$$v(t) = dF0 / dt$$

$$a(t) = dv / dt = d^2F0 / dt^2$$

Both derivatives are computed with NumPy's np.gradient() using the actual timestamps, so velocity is expressed in Hz/s and acceleration in Hz/s². The three features are stacked into a three-channel representation:

$$X = [\ F0,\ v,\ a\ ]$$

This representation allows the CNN to learn from pitch level, pitch movement and changes in pitch movement.

4.2.4 Feature Normalisation and Temporal Segmentation

Before classification, each feature channel c is standardised with the mean μc and standard deviation σc computed from the training data:

$$z = (\ x - \mu c\ ) / \sigma c$$

This keeps the three channels on a comparable scale and supports stable training. The standardised sequence is then divided into fixed-length overlapping frames. Each frame contains 100 time steps, corresponding to one second of audio at the 10 ms CREPE interval, and consecutive frames overlap by 50 time steps (50 % overlap). Frame k covers samples 50k to 50k + 99, so the first frames span samples 0–99, 50–149, 100–199 and 150–249. Each frame contains all three channels:

$$Xk \in \mathbb{R}^{3x100}$$

where 3 is the number of feature channels and 100 the number of time steps in frame k. Overlapping segmentation lets the system analyse local pitch patterns while preserving

continuity between neighbouring frames, and the start time of every frame is retained so that predictions can be mapped back to the recording.

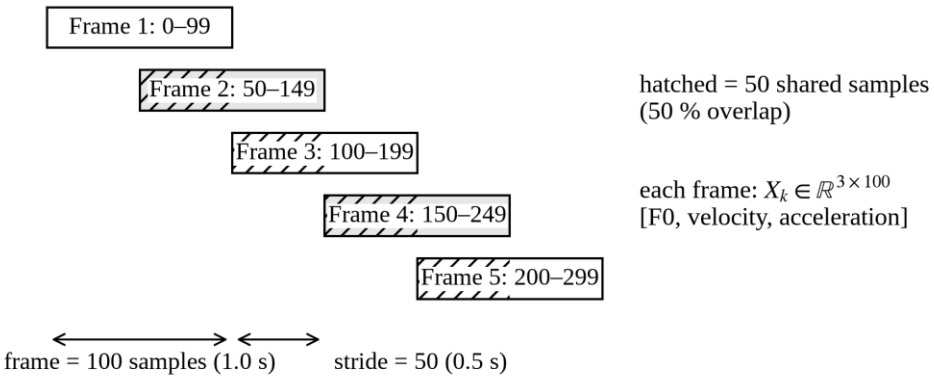


Fig. 4.2 Temporal segmentation into frames of 100 samples with a stride of 50 (50 % overlap)

Fig. 4.2 shows how consecutive one-second frames share 50 samples, so every pitch movement appears in two neighbouring frames.

1. **Frame 1 (samples 0–99):** the first one-second window of the feature sequence.
2. **Frame 2 (samples 50–149):** starts 50 samples later and shares samples 50–99 with Frame 1.
3. **Frame 3 (samples 100–199):** shares samples 100–149 with Frame 2.
4. **Frame 4 (samples 150–249):** shares samples 150–199 with Frame 3.
5. **Frame 5 (samples 200–299):** shares samples 200–249 with Frame 4; the pattern continues to the end of the recording.

4.2.5 CNN-Based Ornament Classification

The standardised frames are passed to a One-Dimensional Convolutional Neural Network. A 1D CNN is suitable for this task because its kernels slide along the time axis and learn local temporal patterns, such as a steady rise, a rapid oscillation or a brief spike, from sequential data. The same architecture is used for every ornament; each model performs binary classification, deciding whether the input frame belongs to its ornament.

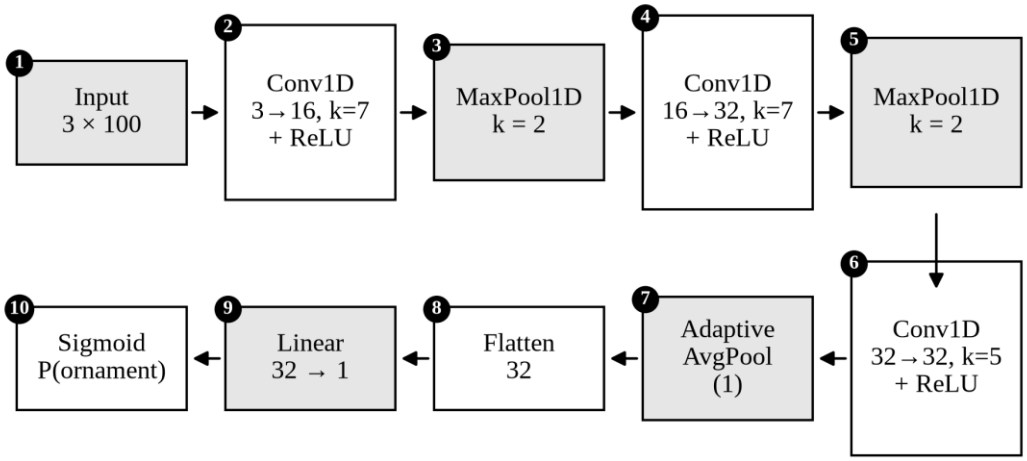


Fig. 4.3 Architecture of the ornament-specific 1D CNN classifier

Fig. 4.3 shows the ten layers of the classifier, from the 3 × 100 input frame to the sigmoid probability.

1. **Input (3 × 100):** one frame of frequency, velocity and acceleration.
2. **Conv1D 3→16, k = 7 + ReLU:** learns 16 short temporal patterns from the three channels.
3. **MaxPool1D (k = 2):** halves the time axis and keeps the strongest responses.
4. **Conv1D 16→32, k = 7 + ReLU:** combines the patterns into 32 higher-level features.
5. **MaxPool1D (k = 2):** halves the time axis again.
6. **Conv1D 32→32, k = 5 + ReLU:** refines the 32 temporal features.
7. **Adaptive average pooling (1):** averages each feature map over time to one value.
8. **Flatten (32):** turns the pooled maps into a 32-value vector.
9. **Linear 32 → 1:** combines the 32 features into one logit.
10. **Sigmoid:** converts the logit into the ornament probability P.

Table 4.1 Configuration of the 1D CNN classifier

Component	Configuration
Input shape	3 channels × 100 time steps
Conv1D layer 1	3 → 16 filters, kernel size 7, ReLU

Component	Configuration
MaxPool1D layer 1	Kernel size 2
Conv1D layer 2	16 → 32 filters, kernel size 7, ReLU
MaxPool1D layer 2	Kernel size 2
Conv1D layer 3	32 → 32 filters, kernel size 5, ReLU
Adaptive average pooling	Output size 1
Flatten / Linear	32 features → 1 output
Output activation	Sigmoid
Trainable parameters	9,153 per classifier
Loss function	Binary cross-entropy with logits (positive-class weighting)
Decision threshold	Ornament-specific τ (τ = 0.78 for Meend)

Table 4.1 lists the layer configuration, parameter count, loss function and decision threshold of each classifier.

4.2.6 Model Training and Optimisation

Each CNN is trained on the labelled frames of its one-vs-rest dataset and learns to separate its ornament from all other frames. During training, mini-batches of frames pass through the network and the predicted probabilities are compared with the binary labels using the binary cross-entropy loss:

$$L = -(1/M)\ \Sigma i\ [\ yi\ \log(\hat{p}i) + (1 - yi)\ \log(1 - \hat{p}i)\ ] \qquad ($$

where M is the number of training samples, yi the true binary label and $\hat{p}i$ the predicted probability. To counter class imbalance, the loss of positive samples is multiplied by the positive-class weight:

$$pos_weight = Nneg / Npos$$

where $Nneg$ and $Npos$ are the numbers of negative and positive training frames. The parameters are updated to minimise the loss, training and validation loss are monitored to detect overfitting, and the decision threshold is selected on the validation set before the model is saved.

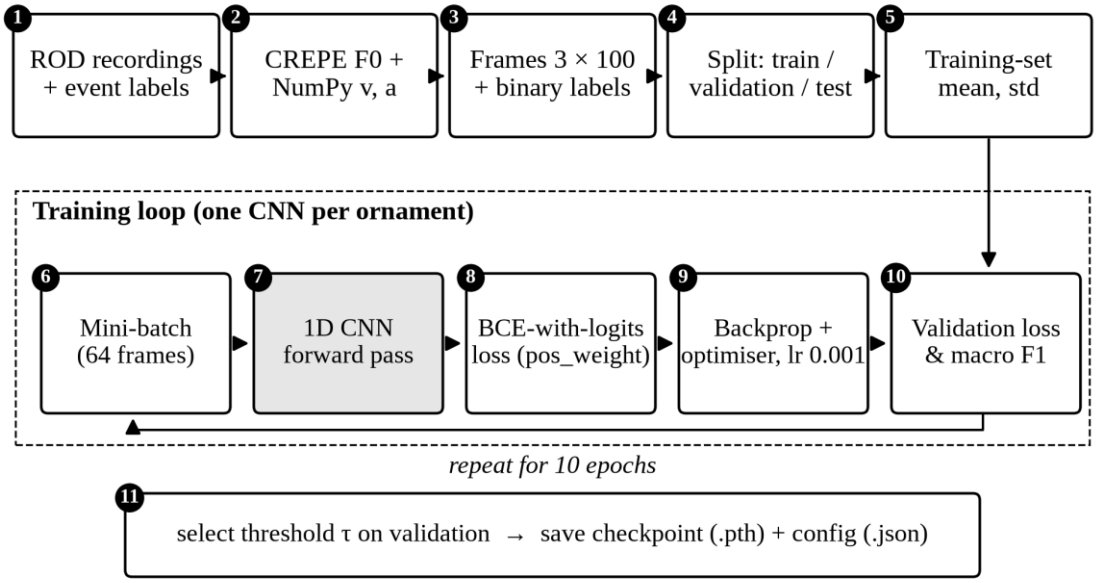


Fig. 4.4 Training workflow of an ornament-specific CNN classifier

Fig. 4.4 shows how labelled ROD frames are split, standardised and used in the mini-batch training loop that produces the saved checkpoint and configuration.

1. **ROD recordings + event labels:** annotated audio used as training material.
2. **CREPE F0 + NumPy v, a:** extracts the three pitch features from every recording.
3. **Frames 3 × 100 + binary labels:** creates one-second frames labelled present or absent for the ornament.
4. **Split: train / validation / test:** separates the frames into the three evaluation sets.
5. **Training-set mean, std:** computes the standardisation statistics from training data only.
6. **Mini-batch (64 frames):** feeds 64 frames at a time to the network.
7. **1D CNN forward pass:** computes a logit for every frame in the batch.
8. **BCE-with-logits loss (pos_weight):** measures the error, weighting the rare positive frames.
9. **Backprop + optimiser, lr 0.001:** updates the network weights to reduce the loss.
10. **Validation loss & macro F1:** checks progress on unseen validation frames after each epoch.
11. **Select threshold → save checkpoint:** chooses τ on validation data and saves the .pth model and .json configuration.

4.2.7 Ornament Recognition and Output

For each frame, the sigmoid output gives the probability that the frame contains the ornament being evaluated. An ornament-specific threshold τ, selected on validation data, converts the probability into a decision:

$$\hat{y} = 1, \ \ if \ \ P(y = 1 \mid X) \geq \tau$$

$$\hat{y} = 0, \ \ if \ \ P(y = 1 \mid X) < \tau$$

The output contains the detected ornament, its probability and the start and end time of the frame. Consecutive positive frames that overlap are merged into one ornament interval. A rule for combining the six classifiers, for frames where more than one classifier is positive or none crosses its threshold, will be finalised during the next semester.

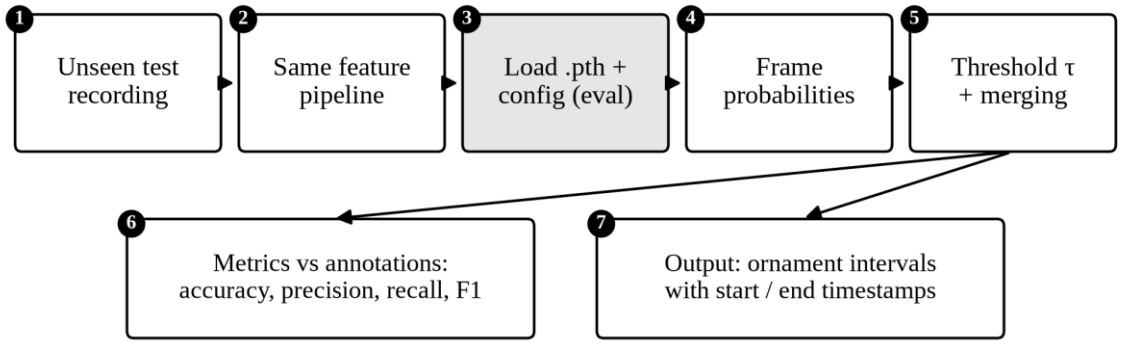


Fig. 4.5 Testing and inference workflow on an unseen recording

Fig. 4.5 shows how an unseen recording passes through the same feature pipeline and the frozen model to produce timestamped ornament decisions and evaluation metrics.

1. **Unseen test recording:** a recording not used during training.
2. **Same feature pipeline:** applies the identical CREPE, NumPy, standardisation and framing steps.
3. **Load .pth + config (eval):** loads the trained weights and settings in evaluation mode.
4. **Frame probabilities:** computes the ornament probability for every frame.
5. **Threshold τ + merging:** applies the threshold and merges overlapping positive frames.
6. **Metrics vs annotations:** compares the decisions with the labels to compute accuracy, precision, recall and F1.

7. **Output: ornament intervals:** reports each detected ornament with its start and end time.

4.3 System Architecture

The complete system architecture is organised into three stages: (A) signal processing and feature extraction, (B) six ornament-specific binary CNN classifiers sharing the same architecture, and (C) decision and output.

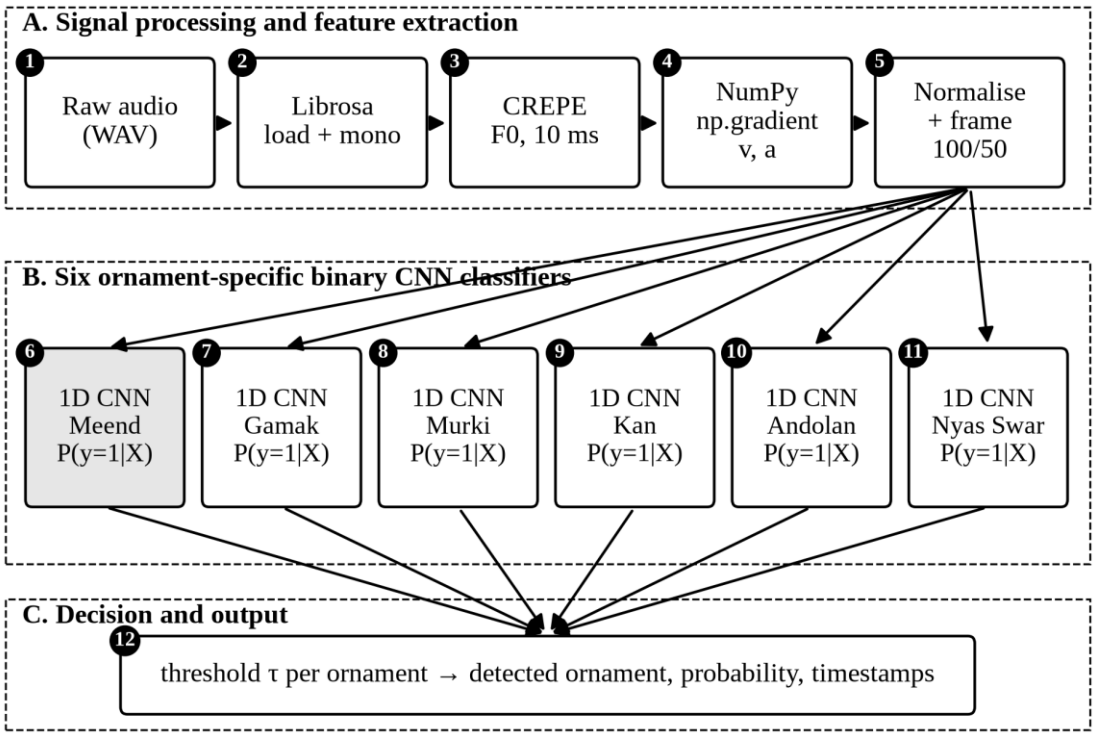


Fig. 4.6 System architecture of the proposed ornament recognition framework

Fig. 4.6 shows how one shared feature-extraction stage feeds six parallel binary classifiers whose outputs are combined in the decision stage.

1. **Raw audio (WAV):** the input recording.
2. **Librosa load + mono:** reads the file and prepares a mono waveform.
3. **CREPE F0, 10 ms:** produces the pitch contour and confidence.
4. **NumPy np.gradient v, a:** derives velocity and acceleration.
5. **Normalise + frame 100/50:** standardises the features and forms overlapping frames.
6. **1D CNN – Meend:** binary classifier for glides between notes.

7. **1D CNN – Gamak:** binary classifier for heavy oscillations.
8. **1D CNN – Murki:** binary classifier for quick note clusters.
9. **1D CNN – Kan:** binary classifier for grace notes.
10. **1D CNN – Andolan:** binary classifier for gentle oscillations.
11. **1D CNN – Nyas Swar:** binary classifier for sustained resting notes.
12. **Threshold τ per ornament:** converts the six probabilities into detected ornaments with probabilities and timestamps.

4.4 Implementation

This section describes how the framework is implemented and evaluated: the dataset used, the hypotheses tested, the training and testing setup, the analysis of the results and the working prototype.

4.4.1 Dataset Description

Raga Ornamentation Detection (ROD) Dataset

The project uses the **Raga Ornamentation Detection (ROD)** dataset released by Kumar, Singh and Arora (IIT Kanpur) [3], [26]. It is one of the first publicly available datasets of Hindustani vocal recordings with expert, time-aligned ornament annotations, which makes it well suited to supervised ornament recognition.

Table 4.2 Summary of the ROD dataset [3], [26]

Property	Details
Genre	Hindustani classical vocal music
Recordings	212 mono WAV files
Total duration	14,693 s (approximately 4.08 hours)
Performers	Two expert vocalists: Singer 1 (108 recordings), Singer 2 (104 recordings)
Rāgas	Bāgeshrī, Bhūpāli, Bhairav and Darbāri
Audio format	44.1 kHz, 32-bit; recorded with an Audio-Technica AT2020 condenser microphone
Accompaniment	Tanpura and tabla recorded on separate tracks; vocal track used for analysis

Property	Details
Metadata	Tonic, tāla and tempo for each recording
Ornament classes	Kan, Meend, Murki, Nyas Swar, Andolan, Gamak (six classes)
Annotation type	Event-based: onset time, offset time and class label; ornaments treated as non-overlapping events
Annotation quality	Two-stage expert annotation; inter-annotator agreement Cohen's κ = 0.79

Table 4.2 lists the size, recording conditions, performers, rāgas and annotation scheme of the ROD dataset.

The recordings are distributed unevenly across rāgas. Bhūpāli is the largest subset (about 148 minutes), whereas Darbāri contributes under five minutes, so rāga-wise imbalance must be considered when splitting the data for training and testing.

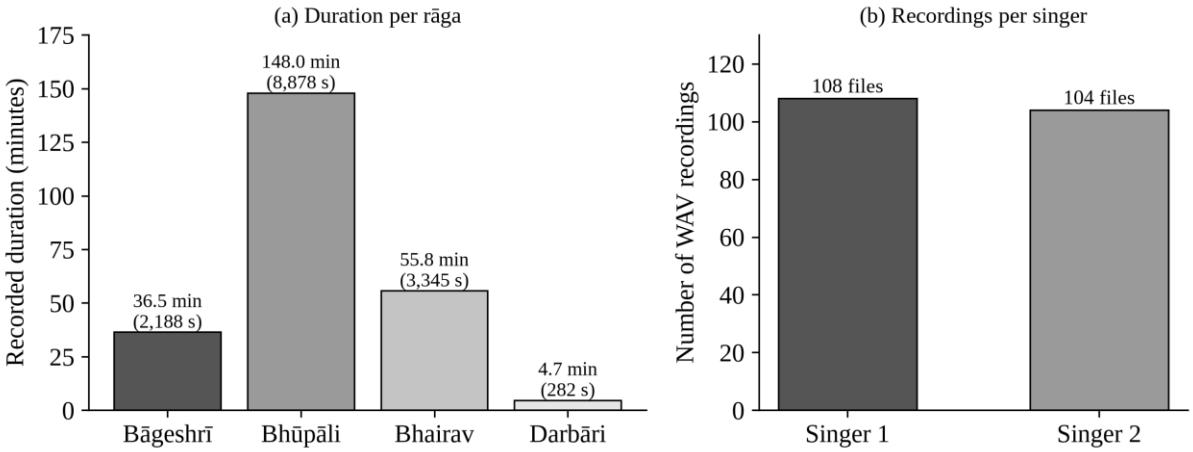


Fig. 4.7 Distribution of the ROD dataset: (a) recorded duration per rāga, (b) recordings per singer

Fig. 4.7 shows that Bhūpāli dominates the recorded duration while the two singers contribute an almost equal number of recordings.

Table 4.3 Duration of ROD recordings per rāga [3]

Rāga	Duration (s)	Duration (min)	Share of dataset
Bāgeshrī	2,188	36.5	14.9 %
Bhūpāli	8,878	148.0	60.4 %
Bhairav	3,345	55.8	22.8 %
Darbāri	282	4.7	1.9 %

Rāga	Duration (s)	Duration (min)	Share of dataset
Total	**14,693**	**244.9**	**100 %**

Table 4.3 gives the exact duration and percentage share of each rāga in the dataset.

Annotation Process

Annotation was carried out in two stages [3]. In the first stage, expert annotators marked the onset, offset and class of every ornament in Audacity. In the second stage, a separate group refined the labels in a human-in-the-loop (active-annotation) tool that displayed the chromagram, the pitch contour, the first-stage labels and the predictions of an intermediate model, so that closely spaced ornaments could be labelled individually. A blind round by an additional group of annotators gave a Cohen's κ of 0.79, indicating substantial agreement.

To keep labels consistent, each ornament was annotated under a duration rule. The rules show why a one-second analysis frame is a suitable unit: short ornaments such as Kan and Murki fit inside a single frame, while longer ornaments such as Andolan and Nyas Swar span one or more frames.

Table 4.4 Duration rules used for annotating each ornament [3]

Ornament	Duration rule	Note
Kan	≤ 0.35 s	Brief grace note
Meend	≥ 0.45 s	Only the glide itself is labelled
Murki	0.40 – 1.00 s	Quick cluster of notes
Nyas Swar	≥ 0.60 s	No upper limit
Gamak	≥ 0.70 s	Heavy oscillation
Andolan	≥ 1.00 s	Slow oscillation

Table 4.4 lists the minimum and maximum annotated duration of every ornament class.

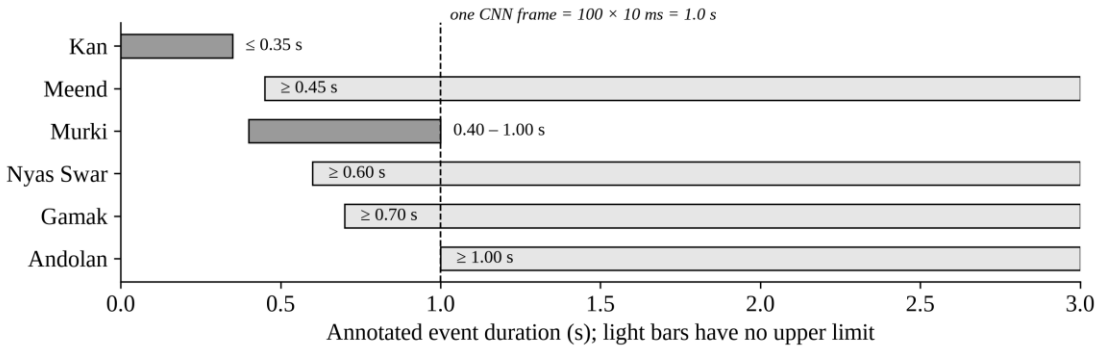


Fig. 4.8 Annotated duration ranges of the six ornaments compared with the one-second CNN frame

Fig. 4.8 compares the duration range of each ornament with the one-second frame analysed by the CNN.

Sample Recording Analysis

To illustrate the data, the recording **001_bageshree_1_Alankar_1_Swar.wav** from the Bāgeshrī subset was processed with the proposed feature-extraction pipeline. The file is a 34.01-second mono recording at 44.1 kHz (32-bit float) of an alankar exercise. The first 14.8 seconds contain only near-silence, and singing continues from about 14.8 s to 32.8 s.

Table 4.5 Measured properties of the sample recording

Property	Value
Duration / sampling rate / channels	34.01 s / 44,100 Hz / 1 (mono)
CREPE model and step size	full capacity, 10 ms step, Viterbi smoothing
F0 samples extracted	3,401 (one every 10 ms)
Voiced samples (confidence ≥ 0.5)	49.3 % of all samples
Contiguous voiced segments	24
F0 range while singing	approximately 175 – 470 Hz
Frames (100 samples, stride 50)	67 frames, model input tensor of shape 67 × 3 × 100

Table 4.5 summarises the signal properties and the number of pitch samples and frames obtained from the sample recording.

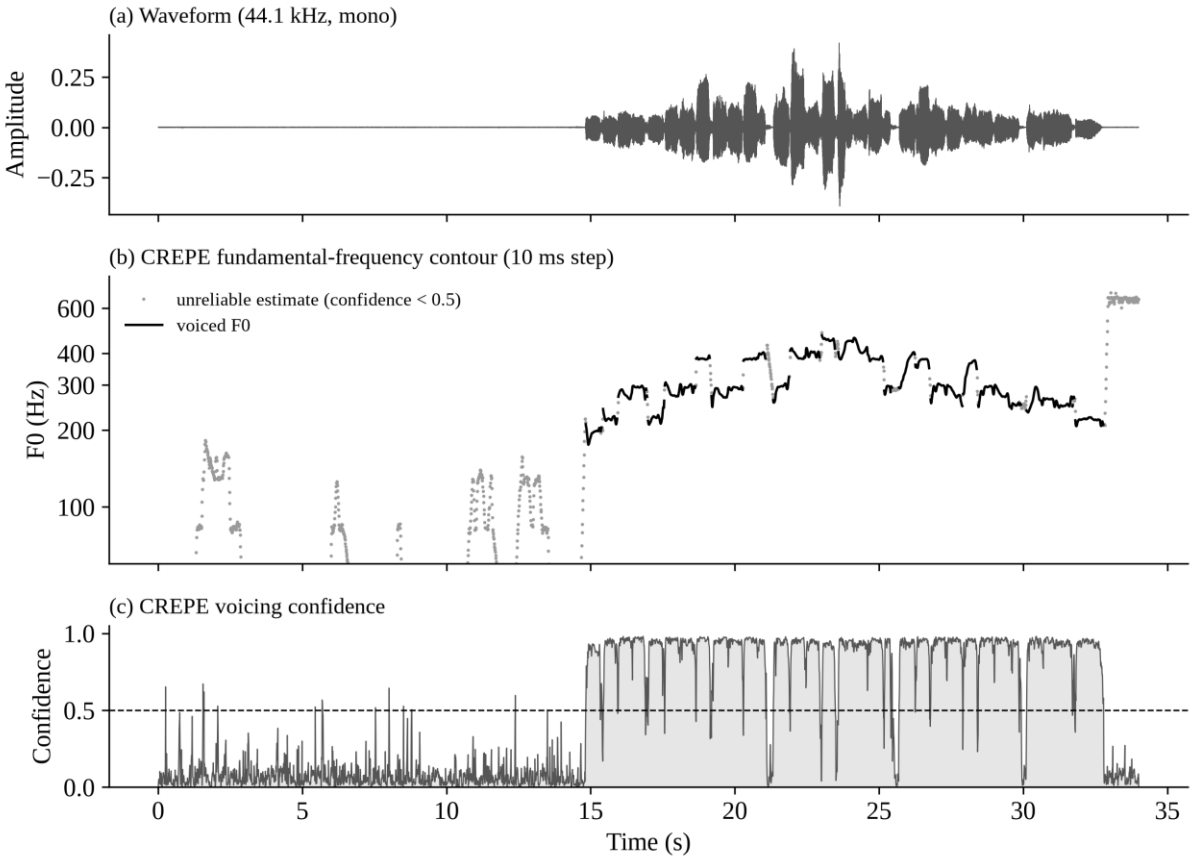


Fig. 4.9 Sample recording (001_bageshree_1_Alankar_1_Swar.wav): (a) waveform, (b) CREPE F0 contour, (c) voicing confidence

Fig. 4.9 shows that reliable pitch appears only after 14.8 s, when the singing starts and CREPE confidence rises above 0.5.

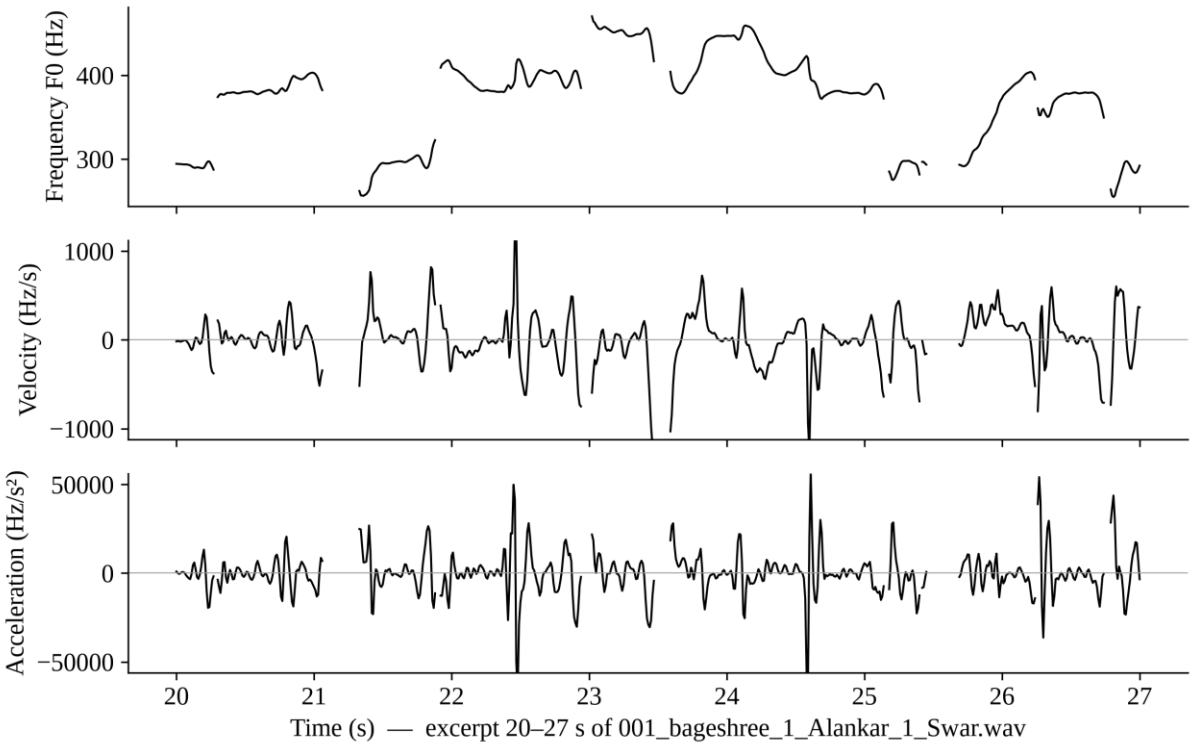


Fig. 4.10 Frequency, velocity and acceleration computed with NumPy for a 7-second excerpt of the sample recording

Fig. 4.10 shows how velocity and acceleration peak at every note transition while staying close to zero on sustained notes.

Preparation of Training Data

For every recording, the vocal track is converted into the three-channel feature sequence and divided into one-second frames. The event annotations (onset, offset and class) are mapped onto these frames so that each frame receives a binary label for the target ornament, *present* (1) or *absent* (0). This produces six one-vs-rest datasets, one per ornament, that share the same frames but differ in their labels.

The resulting datasets are imbalanced because each ornament occupies only a small part of a recording. For the Meend classifier, the positive-class weight stored in its configuration (pos_weight = 7.19) corresponds to roughly 12 % positive frames. Class imbalance is handled with a weighted binary cross-entropy loss and evaluated with macro F1. Following the ROD protocol [3], the data can be split in a general (mixed-singer) setting as well as intra-singer, inter-singer and rāga-specific settings to test generalisation to unseen singers and rāgas.



Fig. 4.11 Class balance of the Meend training frames derived from pos_weight = 7.19

Fig. 4.11 shows that only about 12.2 % of the Meend training frames are positive, which is why a weighted loss and macro F1 are used.

4.4.2 Hypothesis Formulation

Two hypotheses are formulated to test the key design decisions of the framework: the use of pitch dynamics and the use of temporal learning. Both are evaluated with the F1-score on unseen test data.

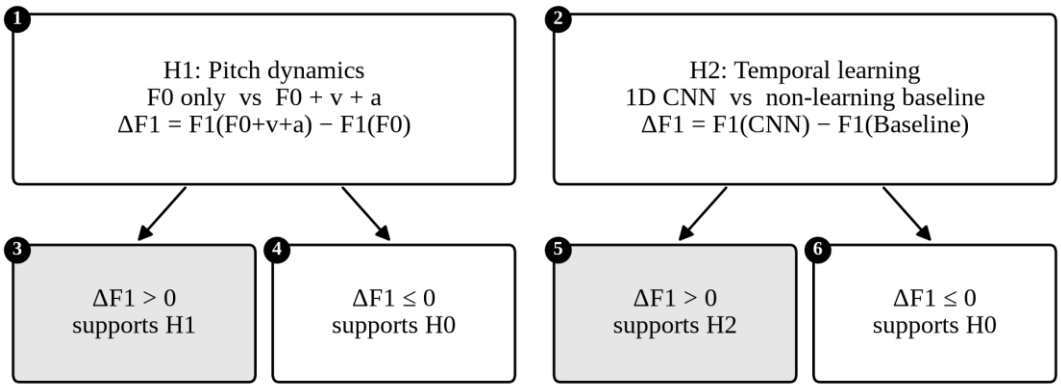


Fig. 4.12 Experimental framework and decision rules for hypotheses H1 and H2

Fig. 4.12 shows the two experimental comparisons and the sign of ΔF1 that decides between each alternative hypothesis and the null hypothesis.

1. **H1: Pitch dynamics:** compares a CNN on F0 only with a CNN on F0 + v + a.
2. **H2: Temporal learning:** compares the 1D CNN with a non-learning baseline.
3. **ΔF1 > 0 (H1):** a positive difference supports H1.
4. **ΔF1 ≤ 0 (H1):** no improvement supports the null hypothesis H0.
5. **ΔF1 > 0 (H2):** a positive difference supports H2.
6. **ΔF1 ≤ 0 (H2):** no improvement supports the null hypothesis H0.

Hypothesis H1: Pitch Dynamics

- **H1 (alternative hypothesis):** F0 dynamics provide additional discriminative information for ornament detection compared with F0 alone.
- **H0 (null hypothesis):** F0 dynamics do not provide additional discriminative information for ornament detection compared with F0 alone.

The test statistic for H1 is:

$$\Delta F1(H1) = F1(F0 + v + a) - F1(F0)$$

Table 4.6 Formulation of hypothesis H1

Element	Definition
Performance measure	F1 = detection performance (F1-score)
Pitch dynamics	Dynamics = { v(t), a(t) }, v(t) = dF0/dt, a(t) = d^2F0/dt^2
H1	F1(F0 + Dynamics) > F1(F0)
H0	F1(F0 + Dynamics) ≤ F1(F0)
Experimental comparison	Model 1: F0 only; Model 2: F0 + v + a
Decision	ΔF1 > 0 → supports H1; ΔF1 ≤ 0 → supports H0

Table 4.6 defines the performance measure, the competing hypotheses, the compared models and the decision rule for H1.

Hypothesis H2: Temporal Learning

- **H2 (alternative hypothesis):** a temporal CNN learns ornament-specific pitch-evolution patterns and can separate ornament patterns on unseen data.
- **H0 (null hypothesis):** a temporal CNN does not learn ornament-specific pitch-evolution patterns; it separates ornament patterns no better than a non-learning baseline.

The test statistic for H2 is:

$$\Delta F1(H2) = F1(CNN) - F1(Baseline)$$

Table 4.7 Formulation of hypothesis H2

Element	Definition
Input	X = [F0, v, a] over 100-step frames
Performance measure	F1 = macro F1-score on unseen test data
H2	F1(CNN) > F1(Baseline)

Element	Definition
H0	$F1(CNN) \leq F1(Baseline)$
Model	Temporal 1D CNN on [F0, v, a]
Baseline	Non-learning classifier (random or majority-class prediction)
Decision	$\Delta F1 > 0 \rightarrow$ supports H2; $\Delta F1 \leq 0 \rightarrow$ supports H0

Table 4.7 defines the input, the CNN and baseline models, and the decision rule used to test H2.

4.4.3 Training and Testing Setup

The experiments follow the training and testing workflows described in Sections 4.2.6 and 4.2.7. The settings used for the Meend classifier and for the functional test on the sample recording are listed below.

Table 4.8 Training and testing configuration

Item	Setting
Training data	ROD frames with one-vs-rest labels for the target ornament
Data partition	Recording-level training, validation and test sets
Input	Standardised [F0, v, a] frames of 3×100 (Eq. 4.5, Eq. 4.6)
Batch size / learning rate / epochs	64 / 0.001 / 10
Loss	BCE with logits, pos_weight = 7.19 (Eq. 4.7, Eq. 4.8)
Decision threshold	$\tau = 0.78$ (Meend)
Evaluation metrics	Macro F1 (primary), precision, recall, accuracy (Eq. 2.1 – 2.5)
Representation study (H1)	Same CNN trained on F0, F0 + v and F0 + v + a; mean ± s.d. over repeated runs
Functional test	Unseen recording 001_bageshree_1_Alankar_1_Swar.wav (34.01 s, 67 frames)

Table 4.8 lists the data, hyper-parameters, loss, threshold and metrics used to train and test the classifiers.

4.4.4 Result Analysis

This section analyses the experimental results and interprets what they reveal about the design of the AI-Based Ornament Detector. Macro F1 is used as the primary metric because the ornament datasets are strongly imbalanced, and accuracy alone can be misleading.

Effect of the Input Representation (H1)

The same CNN was trained with three input representations, and the mean and standard deviation over repeated runs are reported below.

Table 4.9 Effect of the input representation on classification performance (mean ± s.d.)

Representation	Macro F1	Precision	Recall	Accuracy
F0 only	0.4911 ± 0.0223	0.4910 ± 0.0166	0.5030 ± 0.0357	0.8237 ± 0.0123
F0 + Velocity	0.7651 ± 0.0519	0.8522 ± 0.0544	0.7316 ± 0.0841	0.8889 ± 0.0094
F0 + Velocity + Acceleration	**0.8208 ± 0.0298**	0.8362 ± 0.0388	**0.8190 ± 0.0312**	**0.9030 ± 0.0184**

Table 4.9 compares the four evaluation metrics of the CNN for each of the three input representations.

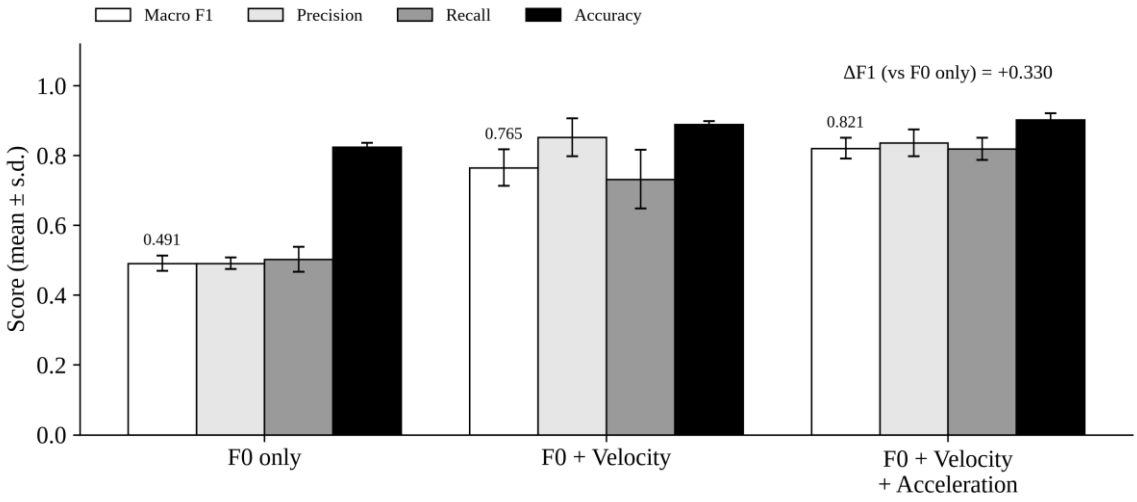


Fig. 4.13 Comparison of input representations for hypothesis H1

Fig. 4.13 shows that every metric improves when velocity and acceleration are added, with the largest gain in macro F1.

F0 only. With pitch values alone, the CNN reaches a macro F1 of 0.491, with precision and recall both close to 0.50. This is no better than a stratified random guess (macro F1 = 0.500,

comparison with non-learning baselines in Section 4.4.4), which means the model cannot separate ornament frames from non-ornament frames using pitch level alone. Its accuracy of 0.824 looks high only because most frames are negative. The result is musically expected: the same swara can be sung as a sustained Nyas Swar or as the end point of a Meend, so the absolute pitch value does not identify the ornament.

Adding velocity. Velocity raises macro F1 by 0.274 to 0.765, a relative improvement of 55.8 %. Precision rises the most, from 0.491 to 0.852, showing that the direction and speed of pitch movement give the model strong evidence of an ornament. Recall improves less (0.732) and has the largest spread across runs (s.d. 0.084), which suggests that ornaments with slow or small movements still produce velocities too low to be recognised reliably.

Adding acceleration. Acceleration adds a further 0.056 to macro F1 (0.821, a 7.3 % relative gain over F0 + velocity) and raises recall from 0.732 to 0.819. Precision falls slightly, by 0.016, but this change is smaller than its run-to-run standard deviation and is not meaningful. Precision and recall are now balanced (0.836 and 0.819), and the standard deviation of macro F1 falls from 0.052 to 0.030, so the model is also more stable. Acceleration describes the curvature of the contour, such as where a glide starts and ends or where an oscillation turns, and this helps the network find ornament frames it previously missed.

The relative improvement used in this analysis is defined as:

$$\textit{Relative gain} = \Delta F1 \,/\, F1(\textit{reference})$$

Table 4.10 Incremental change in each metric when pitch dynamics are added

Change	Macro F1	Precision	Recall	Accuracy
F0 → F0 + v	+0.274 (+55.8 %)	+0.361	+0.229	+0.065
F0 + v → F0 + v + a	+0.056 (+7.3 %)	−0.016	+0.087	+0.014
F0 → F0 + v + a (ΔF1 for H1)	**+0.330 (+67.1 %)**	+0.345	+0.316	+0.079

Table 4.10 lists the gain contributed by velocity, by acceleration and by both together for every metric.

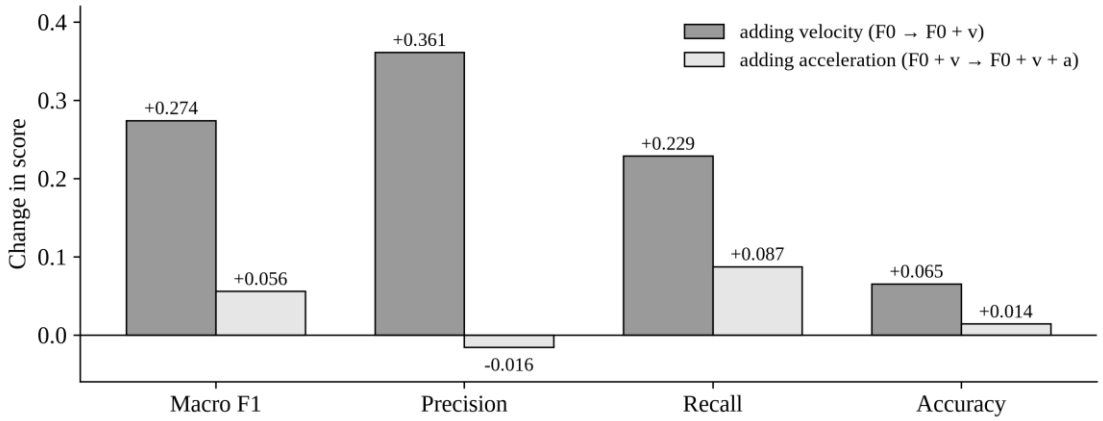


Fig. 4.14 Incremental change in each metric contributed by velocity and by acceleration

Fig. 4.14 shows that velocity contributes most of the improvement in precision, while acceleration mainly improves recall.

Decision on H1. The test statistic of Eq. 4.11 is $\Delta F1 = 0.8208 - 0.4911 =$ **+0.330 > 0**, and the difference is more than ten times the largest run-to-run standard deviation of either model. The experiment therefore **supports H1**: pitch dynamics carry discriminative information that F0 alone does not. Accuracy rises by only 0.079 over the same comparison, which confirms that accuracy hides most of the real improvement on imbalanced ornament data. A formal significance test across folds will be reported next semester once the number of runs is fixed.

Comparison with Non-Learning Baselines (H2)

A non-learning baseline needs no training, so its expected scores follow directly from the class balance. For a positive rate $p \approx 0.122$ (the Meend class balance of Fig. 4.11), a classifier that always predicts "absent" obtains a positive-class F1 of 0 and a negative-class F1 of $2(1 - p) / (2 - p) \approx 0.935$, giving a macro F1 of 0.468. A random 50/50 guess gives 0.417, and a stratified random guess that predicts "present" with probability p gives 0.500.

Table 4.11 Macro F1 of non-learning baselines and CNN models

Model	Learns from data	Macro F1	Accuracy	ΔF1 vs best baseline
Random guess (50/50)	No	0.417	0.500	−0.083
Majority class (always absent)	No	0.468	0.878	−0.032
Stratified random guess	No	0.500	0.786	0.000
CNN on F0 only	Yes	0.491	0.824	−0.009

Model	Learns from data	Macro F1	Accuracy	ΔF1 vs best baseline
CNN on F0 + v + a	Yes	**0.821**	**0.903**	**+0.321**

Table 4.11 compares the macro F1 and accuracy of the three non-learning baselines with the two CNN models.

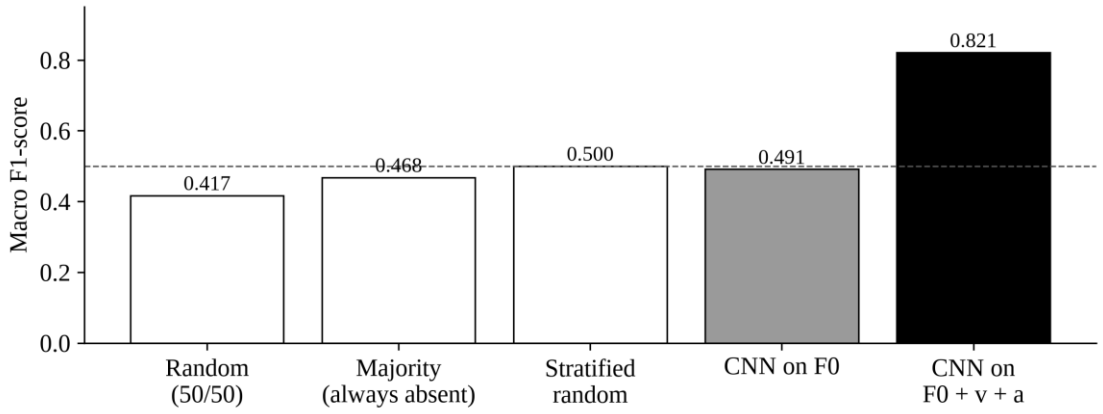


Fig. 4.15 Macro F1 of non-learning baselines compared with the CNN models

Fig. 4.15 shows that only the CNN with pitch dynamics clearly exceeds the 0.5 level reached by guessing.

The CNN trained on F0 + v + a exceeds the best baseline by ΔF1 = +0.321, which gives **indicative support for H2**. The comparison also shows that learning alone is not enough: the CNN trained on F0 only does not beat the stratified baseline. The temporal CNN becomes useful only when the input describes how pitch moves, so the two hypotheses support each other. The majority-class baseline reaches an accuracy of 0.878, higher than the F0-only CNN, which again shows why macro F1 is the deciding metric. Because the baseline values assume the Meend class balance, the formal H2 test on held-out singers and rāgas for all six ornaments is planned for the next semester.

Inference on an Unseen Test Recording

The trained Meend classifier was applied to the unseen sample recording described in Section 4.4.1 as a functional test of the complete pipeline. Of its 67 frames, 31 consist mostly of silence and are excluded, which leaves 36 eligible frames. Ornament labels for this file were not available, so the test checks the behaviour of the model rather than its accuracy.

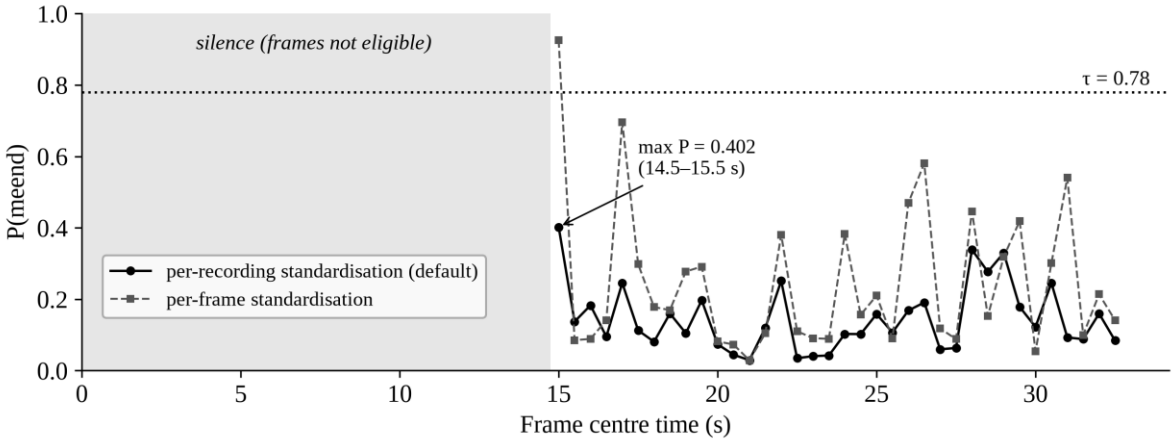


Fig. 4.16 Frame-wise Meend probability on the test recording under two standardisation settings

Fig. 4.16 shows that, with per-recording standardisation, every eligible frame stays below the 0.78 threshold, and the highest probability appears at the onset of singing.

Table 4.12 Highest-probability frames of the test recording

Frame	Time (s)	Voiced fraction	P (per-recording)	P (per-frame)
29	14.5 – 15.5	0.61	0.402	0.925
55	27.5 – 28.5	0.94	0.339	0.446
57	28.5 – 29.5	1.00	0.330	0.319
56	28.0 – 29.0	0.96	0.277	0.152
43	21.5 – 22.5	0.97	0.251	0.381

Table 4.12 lists the five frames with the highest Meend probability and their probabilities under both standardisation settings.

With the default per-recording standardisation, no frame reaches the threshold, and the mean probability of the eligible frames is 0.145. The model nevertheless ranks the frames sensibly. The highest probability (0.402) occurs at 14.5–15.5 s, where the singer enters with a rising scoop from about 176 Hz to 198 Hz followed by a jump to about 230 Hz. The next-highest frames, at 27.5–29.5 s, contain rising transitions between swaras. Frames with steady notes receive low probabilities. Because the recording is an alankar exercise made mostly of stepwise swaras, few full meends are expected, so the low probabilities are plausible.

When the same frames are standardised per frame instead, frame 29 rises to 0.925 and would be detected, and the mean probability rises to 0.247. The model's decisions therefore depend

strongly on the statistics used in Eq. 4.5. This supports using the stored training-set mean and standard deviation at inference time, rather than statistics computed from each test recording.

Robustness to Preprocessing Choices

To measure this sensitivity, the classifier was re-run on the same CREPE output under three standardisation modes, three voicing-confidence thresholds and two gradient edge orders.

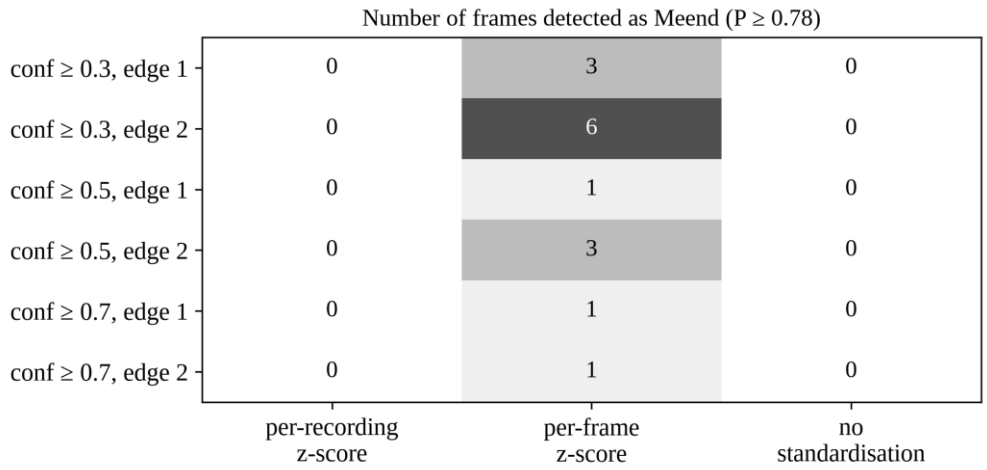


Fig. 4.17 Number of frames detected as Meend under different preprocessing settings

Fig. 4.17 shows that detections appear only with per-frame standardisation and that their number varies with the voicing threshold and gradient edge order.

Without standardisation, no frame is detected in any setting, and the logits reach values as low as −35,849. This confirms that the network was trained on standardised inputs and cannot be used on raw Hz values. Per-recording standardisation produced no detections in any of the six settings, whereas per-frame standardisation produced between one and six detections depending on the voicing threshold and edge order. Decisions close to the threshold are therefore sensitive to preprocessing. The frame at 31.5–32.5 s is detected under four of the six per-frame settings, the most consistent of any frame.

Summary of Findings

Table 4.13 Summary of the result analysis

Finding	Evidence	Implication
Pitch dynamics are essential	Macro F1 0.491 → 0.821 (ΔF1 = +0.330)	H1 supported; keep the three-channel [F0, v, a] input

Finding	Evidence	Implication
Velocity drives precision, acceleration drives recall	Precision +0.361 with v; recall +0.087 with a	Both derivatives are needed for balanced detection
F0 alone is at chance level	Macro F1 0.491 vs stratified baseline 0.500	Absolute pitch does not identify an ornament
CNN with dynamics beats non-learning baselines	$\Delta F1 = +0.321$ over the best baseline	Indicative support for H2; formal test next semester
Accuracy is misleading	Majority baseline accuracy 0.878 > F0-only CNN 0.824	Report macro F1 as the primary metric
Decisions depend on standardisation	Frame 29: P = 0.402 (per-recording) vs 0.925 (per-frame)	Store and reuse training-set mean and std

Table 4.13 summarises each finding together with its supporting evidence and its implication for the next stage of the work.

4.4.5 Prototype Implementation

An end-to-end prototype has been implemented for the Meend classifier to demonstrate the complete pipeline. A FastAPI backend loads the trained PyTorch checkpoint and its JSON configuration, runs CREPE pitch extraction, computes velocity and acceleration with NumPy, builds the 100-step frames and returns frame-level probabilities and detected intervals with timestamps. A React dashboard displays the pitch contour, the motion features, the frame segmentation, the model configuration and the frame-wise probabilities with the decision threshold, and exports the results as CSV, JSON and an HTML report.

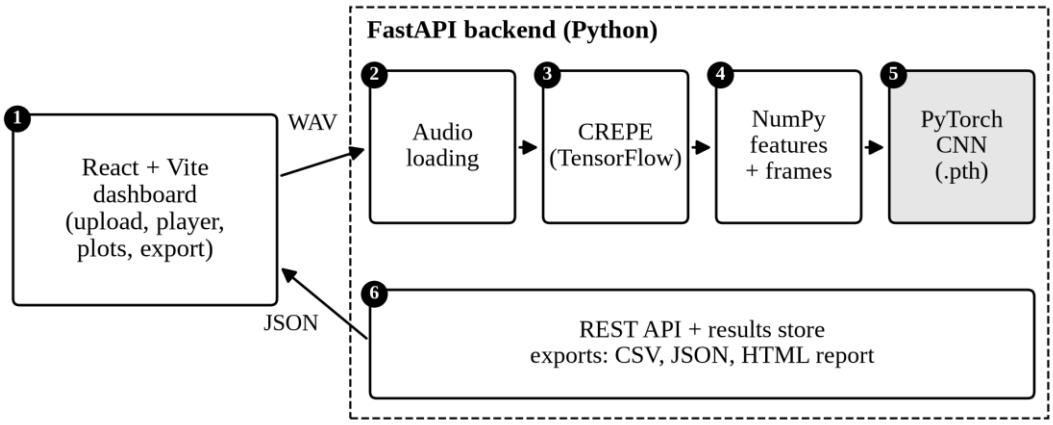


Fig. 4.18 Deployment architecture of the prototype

Fig. 4.18 shows how the React dashboard sends the WAV file to the FastAPI backend, which runs the audio, CREPE, NumPy and PyTorch modules and returns the results as JSON.

1. **React + Vite dashboard:** web interface for uploading audio, playback, plots and export.
2. **Audio loading:** reads the uploaded WAV file on the server.
3. **CREPE (TensorFlow):** extracts the pitch contour.
4. **NumPy features + frames:** computes velocity, acceleration and the 100-step frames.
5. **PyTorch CNN (.pth):** runs the trained Meend classifier on every frame.
6. **REST API + results store:** returns the results as JSON and provides CSV, JSON and HTML exports.

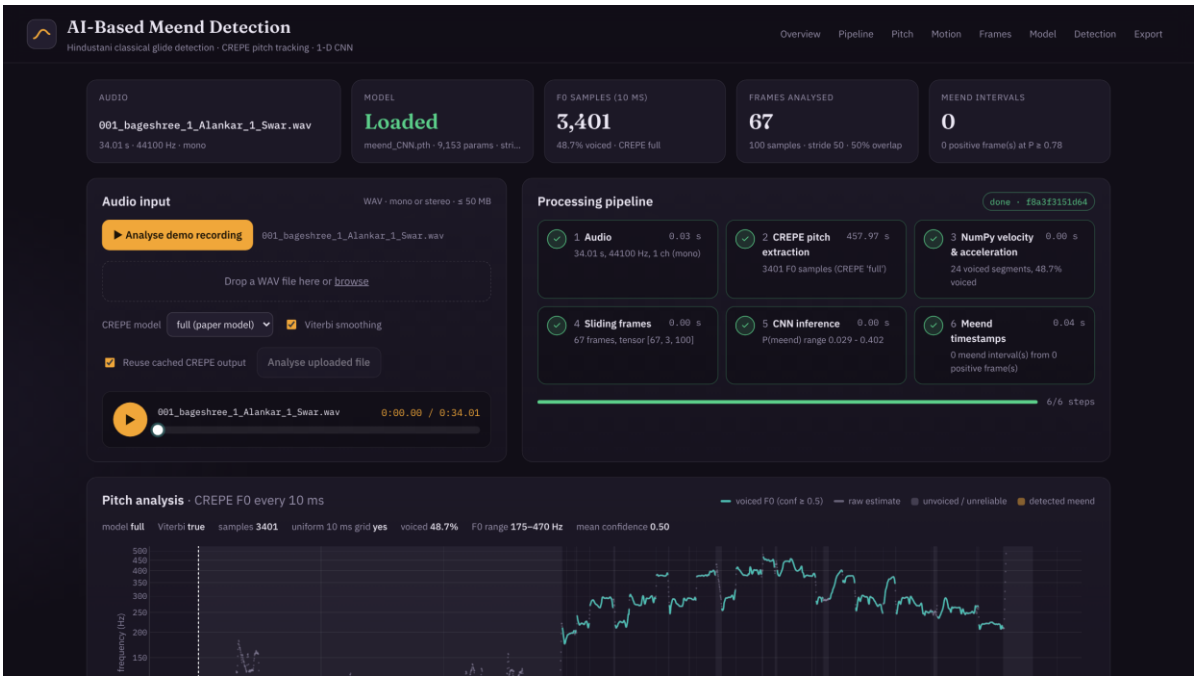


Fig. 4.19 Dashboard of the prototype analysing a ROD recording

Fig. 4.19 shows the dashboard after a completed analysis, with the six pipeline steps, the audio player and the pitch plot.

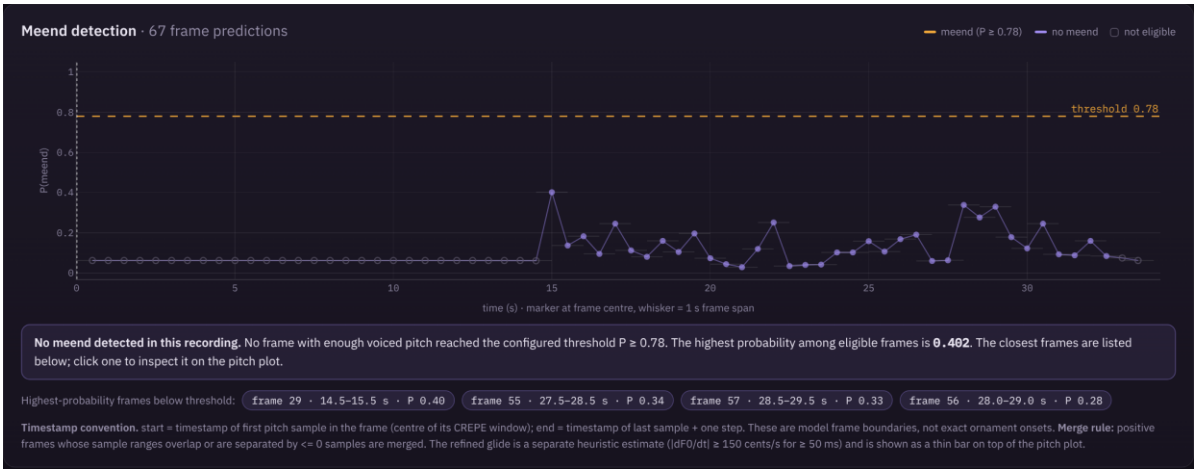


Fig. 4.20 Detection panel of the prototype showing frame-wise probabilities on the test recording

Fig. 4.20 shows the frame-wise Meend probabilities, the 0.78 threshold and the highest-probability frames reported by the prototype for the test recording.

5. Plan for Next Semester

The next semester will focus on completing, improving and extending the CNN-based ornament recognition framework. The planned work is listed below.

1. **Completion of ornament recognition:** extend the pipeline from the Meend prototype to all six ornaments: Meend, Gamak, Murki, Kan, Andolan and Nyas Swar [1]–[3].
2. **Dataset preparation and expansion:** improve the balance, diversity and annotation quality of samples across ornament categories, and evaluate on additional concert recordings to test generalisation [3], [26].
3. **CNN model optimisation:** experiment with CNN configurations, training parameters and decision thresholds to improve classification performance [15], [16], [21].
4. **Feature representation analysis:** analyse the contribution of frequency, velocity and acceleration for ornaments with similar pitch characteristics, extending the H1 study to each ornament [1], [2].
5. **Complete audio analysis:** process full-length recordings and locate ornament occurrences in continuous audio rather than in isolated segments [3], [19], [22].
6. **Performance evaluation:** evaluate all classifiers with accuracy, precision, recall, F1-score and confusion matrices, report training and validation loss curves, and complete the H2 test against non-learning baselines on unseen singers and rāgas.
7. **System integration:** develop the user-facing application from the current prototype so that users can upload recordings and view the recognised ornaments with timestamps.

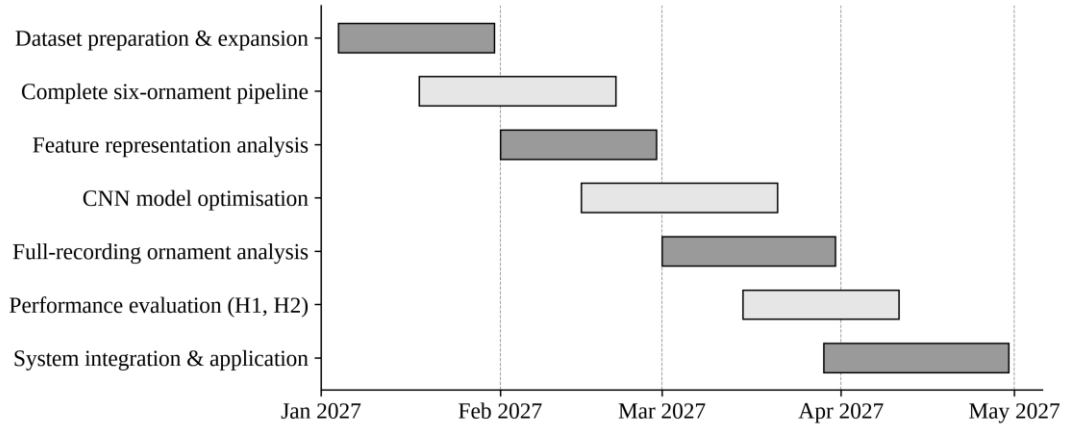


Fig. 5.1 Timeline of the work planned for the next semester

Fig. 5.1 shows the start and end of each planned activity between January and April 2027.

References

[1] Pratyush, “Analysis and Classification of Ornaments in North Indian (Hindustani) Classical Music,” Master’s thesis, Universitat Pompeu Fabra, Barcelona, Spain, 2010.

[2] K. Banerjee, A. Patranabis, R. Sengupta, and D. Ghosh, “Ornamentation in Hindustani Vocal Music,” in Proc. 27th Int. Symp. Frontiers of Research in Speech and Music (FRSM), Springer, 2024, pp. 125–135.

[3] S. Kumar, P. Singh, and V. Arora, “Recognizing Ornaments in Vocal Indian Art Music With Active Annotation,” IEEE/ACM Trans. Audio, Speech, and Language Processing, vol. 34, pp. 220–229, 2026, doi: 10.1109/TASLPRO.2025.3639738. Preprint: arXiv:2505.04419.

[4] C. Gupta and P. Rao, “Objective Assessment of Ornamentation in Indian Classical Singing,” in Speech, Sound and Music Processing: Embracing Research in India (CMMR 2011 / FRSM 2011), LNCS vol. 7172, Springer, 2012.

[5] A. Vidwans, K. K. Ganguli, and P. Rao, “Classification of Indian Classical Vocal Styles from Melodic Contours,” in Proc. 2nd CompMusic Workshop, Istanbul, Turkey, 2012.

[6] S. S. Miryala, K. Bali, R. Bhagwan, and M. Choudhury, “Automatically Identifying Vocal Expressions for Music Transcription,” in Proc. Int. Society for Music Information Retrieval Conf. (ISMIR), Curitiba, Brazil, 2013.

[7] S. Gulati, J. Serrà, K. K. Ganguli, S. Şentürk, and X. Serra, “Time-Delayed Melody Surfaces for Rāga Recognition,” in Proc. Int. Society for Music Information Retrieval Conf. (ISMIR), New York, USA, 2016.

[8] S. Giraldo and R. Ramírez-Meléndez, “A Machine Learning Approach to Ornamentation Modeling and Synthesis in Jazz Guitar,” in Machine Learning and Music Generation, Routledge, 2018, pp. 21–40.

[9] Y. Yamamoto, J. Nam, and H. Terasawa, “Analysis and Detection of Singing Techniques in Repertoires of J-Pop Solo Singers,” in Proc. Int. Society for Music Information Retrieval Conf. (ISMIR), 2022.

[10] Y. Yamamoto, J. Nam, and H. Terasawa, “Deformable CNN and Imbalance-Aware Feature Learning for Singing Technique Classification,” arXiv preprint arXiv:2206.12230, 2022.

[11] Y. Yamamoto, J. Nam, and H. Terasawa, “PrimaDNN: A Characteristics-Aware DNN Customization for Singing Technique Detection,” in Proc. European Signal Processing Conf. (EUSIPCO), IEEE, 2023, pp. 406–410.

[12] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, “CREPE: A Convolutional Representation for Pitch Estimation,” in Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP), 2018, pp. 161–165.

[13] J. Wilkins, P. Seetharaman, A. Wahl, and B. Pardo, “VocalSet: A Singing Voice Dataset,” in Proc. Int. Society for Music Information Retrieval Conf. (ISMIR), 2018, pp. 468–474.

[14] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and Music Signal Analysis in Python,” in Proc. 14th Python in Science Conf. (SciPy), 2015, pp. 18–25.

[15] K. J. Piczak, “Environmental Sound Classification with Convolutional Neural Networks,” in Proc. IEEE Int. Workshop on Machine Learning for Signal Processing (MLSP), 2015.

[16] J. Salamon and J. P. Bello, “Deep Convolutional Neural Networks and Data Augmentation for Environmental Sound Classification,” IEEE Signal Processing Letters, vol. 24, no. 3, pp. 279–283, 2017.

[17] E. Çakır, G. Parascandolo, T. Heittola, H. Huttunen, and T. Virtanen, “Convolutional Recurrent Neural Networks for Polyphonic Sound Event Detection,” IEEE/ACM Trans. Audio, Speech, and Language Processing, vol. 25, no. 6, pp. 1291–1303, 2017.

[18] C.-C. Kao, W. Wang, M. Sun, and C. Wang, “R-CRNN: Region-Based Convolutional Recurrent Neural Network for Audio Event Detection,” arXiv preprint arXiv:1808.06627, 2018.

[19] C. Lea, R. Vidal, A. Reiter, and G. D. Hager, “Temporal Convolutional Networks: A Unified Approach to Action Segmentation,” in Proc. European Conf. Computer Vision (ECCV) Workshops, 2016, pp. 47–54.

[20] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, “Temporal Convolutional Networks for Action Segmentation and Detection,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2017, pp. 156–165.

[21] S. Bai, J. Z. Kolter, and V. Koltun, “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling,” arXiv preprint arXiv:1803.01271, 2018.

[22] A. Mesaros, T. Heittola, T. Virtanen, and M. D. Plumbley, “Sound Event Detection: A Tutorial,” IEEE Signal Processing Magazine, vol. 38, no. 5, pp. 67–83, 2021.

[23] S. Aswale, P. C. Shrivastava, R. Ranjan, and S. Shende, “Indian Classical Music Recognition Using Deep Convolution Neural Network,” Int. J. Electrical and Electronics Research, vol. 12, no. 1, pp. 73–82, 2024.

[24] D. P. Shah, N. M. Jagtap, P. T. Talekar, and K. Gawande, “Raga Recognition in Indian Classical Music Using Deep Learning,” in Lecture Notes in Computer Science, Springer, 2021.

[25] “Gamaka Synthesis for Kalpitha Swaras in Carnatic Music,” in Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing Workshops (ICASSPW), 2025, doi: 10.1109/ICASSPW65056.2025.11011251.

[26] S. Kumar, P. Singh, and V. Arora, “Raga Ornamentation Detection (ROD) Dataset,” Zenodo, 2025. [Online]. Available: https://zenodo.org/records/17851882